RESEARCH PROTOCOL & IMPLEMENTATION SNAPSHOT

LLM-Generated Tests for C# Libraries

A reproducible empirical check of the relationship between code coverage and mutation score in .NET unit tests.

This document records what has been designed and implemented so far. It deliberately contains no fabricated experimental results.

01 / EXECUTIVE SUMMARY

What this project is about

The study asks whether tests generated by one configurable large language model can achieve high structural coverage while missing faults exposed by mutation testing. Generated tests are compared with each library's existing human-written tests, and a single feedback round is used to test whether surviving mutants help the model improve.

8	2	3×
target libraries	prompt strategies	repetitions per class and strategy

> **Current status.** The repository, candidate screening record, pipeline skeleton, report template, and analysis scripts are in place. Coverage, mutation, compile-rate, and pass-rate results remain pending until the runtime environment and model credentials are available.

Research questions

- **RQ1:** How do coverage and mutation score differ between LLM-generated and human-written tests?
- **RQ2:** Does one feedback round improve mutation score over the basic prompt?
- **RQ3:** What categories of mutants survive LLM-generated tests?

02 / STUDY DESIGN

A deliberately bounded experiment

Subjects

Small or medium pure-logic open-source C# libraries are screened for a permissive license, .NET 8 compatibility, and an existing xUnit or NUnit test suite. Each subject is pinned to a commit and only three to five source files are mutated.

Model configuration

The model name, endpoint, and API key are supplied through environment variables. Temperature is fixed at `0.2`. Secrets are never stored in source, prompts, CSV files, or git.

Two strategies

A / BASIC The class source and dependency signatures are supplied with a request to write xUnit tests.

B / FEEDBACK The same initial request is followed by one additional round containing surviving mutants from round one.

Replication

Each target class and strategy is generated three times. Raw prompts, raw responses, metadata, validation output, coverage reports, and mutation reports are retained.

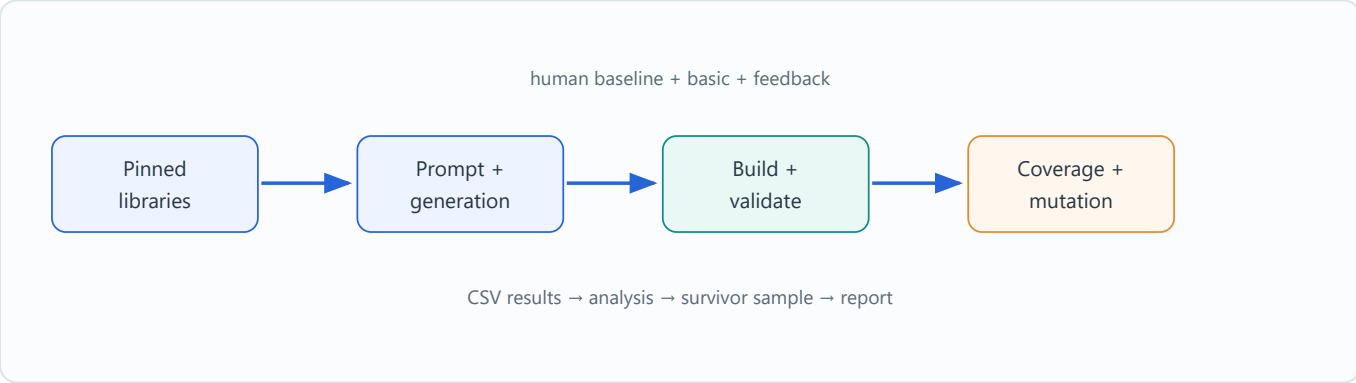


Figure 1. Planned data flow. Failures and timeouts remain part of the recorded data rather than being silently excluded.

03 / CANDIDATE SCREENING

Selected subjects and size snapshot

The initial screening identified eight practical subjects for the study. Production LOC below is a deterministic source snapshot excluding test, build, and generated directories. It is a subject-selection measure, not an experimental outcome.

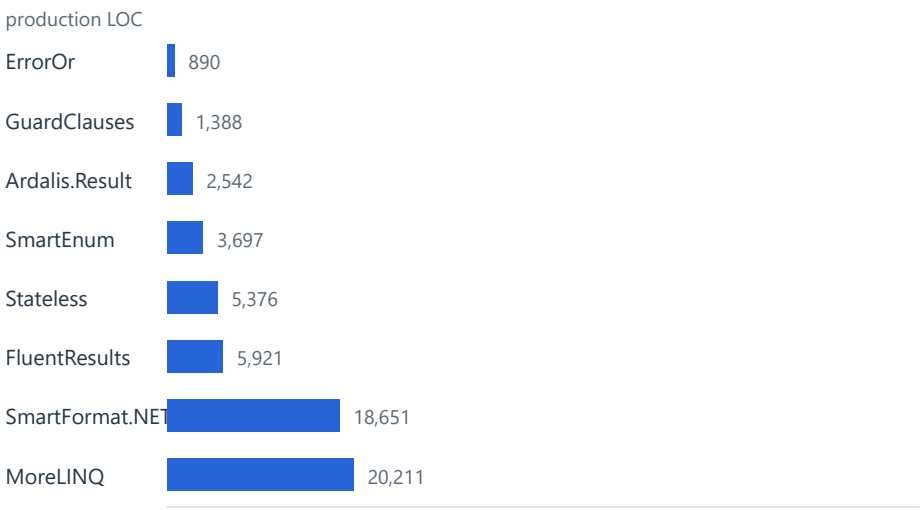


Figure 2. Production source-size snapshot at the pinned shallow-clone commits. Final inclusion still requires actual build and test verification.

Library	License	Tests	Selection note
ErrorOr	MIT	xUnit	Small pure-logic candidate
Ardalis.GuardClauses	MIT	xUnit	Small pure-logic candidate
Ardalis.Result	MIT	xUnit	Small pure-logic candidate
Ardalis.SmartEnum	MIT	xUnit	Small pure-logic candidate
Stateless	MIT	xUnit	Small pure-logic candidate
FluentResults	MIT	xUnit	Small pure-logic candidate
SmartFormat.NET	MIT	NUnit	Medium fallback candidate
MoreLINQ	MIT	NUnit	Medium fallback candidate

What has been implemented

Repository structure

- `libs.csv` — subjects, commits, licenses, size, and mutation-file record.
- `src/` — separate commands for selection, generation, validation, measurement, feedback, analysis, and survivor sampling.
- `data/raw/` — prompts, model responses, and tool reports.
- `data/results/` — analysis-ready CSV outputs.
- `report/` — report skeleton and this overview.

Integrity controls

- Commits are recorded rather than tracking moving branches.
- API keys are read only from environment variables.
- Raw prompts and responses are retained.
- Compile failures, failing tests, and timeouts are recorded.
- External library clones and build artifacts are excluded from git.

Planned analysis outputs

The analysis stage will produce coverage, mutation score, compile rate, pass rate, per-library breakdowns, means and standard deviations across repetitions, a coverage-versus-mutation scatter plot, and a stratified CSV sample of 60 surviving mutants with an empty manual-label column.

> **Important boundary.** The current repository is a reproducible protocol and implementation snapshot, not a completed empirical result. Any conclusion about a coverage–mutation gap must wait for successful execution of the full pipeline.

05 / LIMITATIONS AND NEXT STEPS

What still needs to happen

1. Run the selected repositories through real .NET 8 build and test verification.
2. Record three to five mutation targets per included library.
3. Configure one model and run the basic and feedback generations three times per class.
4. Validate generated tests, retaining all compile and test failures in the results.
5. Run Coverlet and Stryker.NET, then generate the tables, plot, and survivor sample.
6. Manually classify the sampled surviving mutants and complete the report.

Threats to validity

- Public libraries may have appeared in the model's training data.
- A single model and a small library sample limit generalization.
- Mutation score depends on Stryker.NET's available mutation operators.
- Survivor classification is manual and performed by one person.

Prepared as a transparent project overview. No empirical result is reported in this document before the measurement pipeline has run.